CSC 251 Lecture 09: More File I/O and Processing Files

Reading and Writing Numeric Data

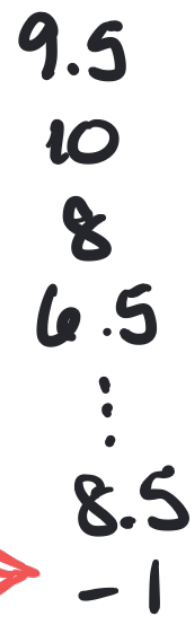
The `read()`, `readline()`, and `readlines()` methods read file input as strings. We must convert numerical values to use them.

Example 1a: Recall the quiz average calculation with the -1 sentinel? Let's incorporate file I/O, assuming the values are contained in a file, each value on its own line. Note the algorithm!

```
with open("grades1.txt") as infile:
    sum = 0
    count = 0
    quiz = float(infile.readline())
    while quiz != -1:
        sum += quiz
        count += 1
        quiz = float(infile.readline())

print(f"Average = {sum/count:.2f}")
```

read
check
process
read



To write the output to a file numeric input must be converted to a string.

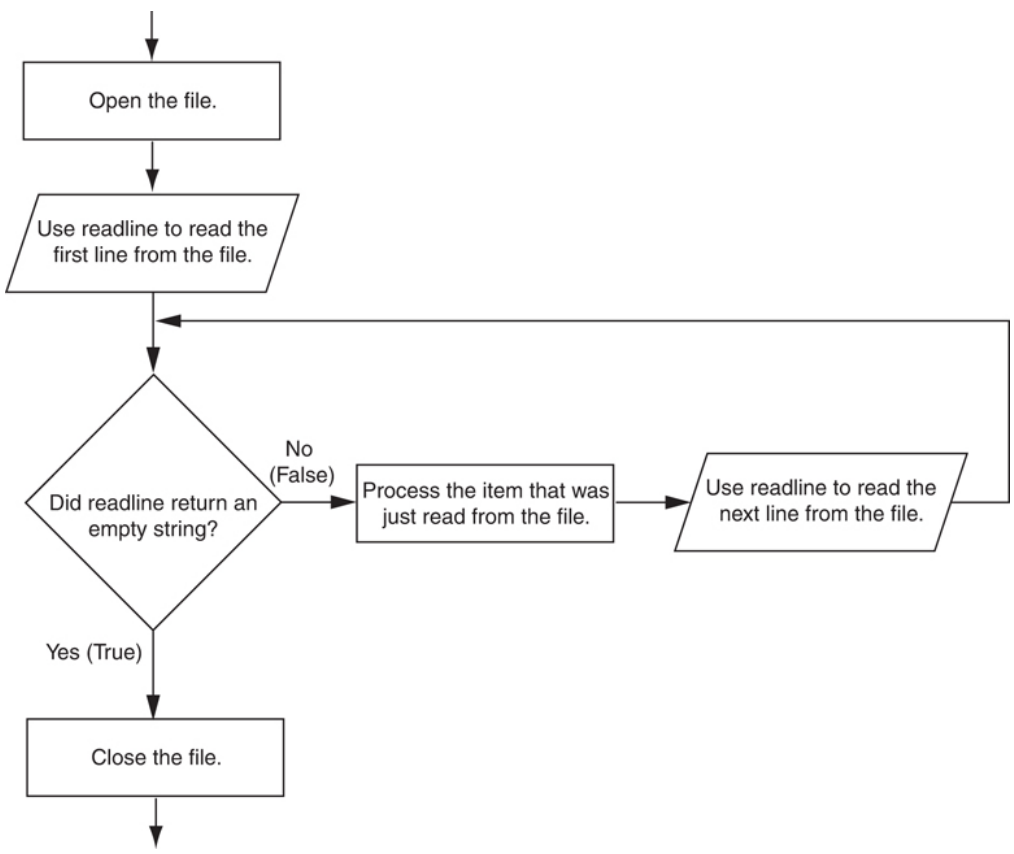
Notice the f-string has already prepared the output in string format, so the numerical value is already converted!

If not using f-strings, convert the numeric value to a string before concatenating the newline character:

```
outfile.write(str(sum/count) + "\n")
```

When/if the data is read back into your program, it can be converted back to a numeric format then.

Whether we read until the sentinel is read or until the end of the datafile, neither the -1 nor the empty string should be included in our calculation. To avoid this, we follow the same order that we did before:



READ
TEST
PROCESS
READ NEXT
(return to loop test)

Example 1b: To read in quiz scores until the end of the file:

```
with open("grades1.txt") as infile:
    sum = 0
    count = 0
    quiz = float(infile.readline())
    while quiz != '':
        sum += quiz
        count += 1
        quiz = float(infile.readline())

print(f"Average = {sum/count:.2f}")
```

Example 1c: Python makes this even easier:

reads til EOF

```
with open("grades1.txt", "r") as infile:
    sum = 0
    count = 0
    for line in infile:
        quiz = float(line)
        sum += quiz
        count += 1

print(f"Average = {sum/count:.2f}")
```

check not done yet
read
process

The algorithm changes!

Reading Records from a Data File

Suppose a file contains a student's name, ID number, and a list of credit hours earned for the last 4 semesters —for numerous students.

```
Steve Wozniak
135790
17
16
18
16
Steve Jobs
100100
17
14
15
16
Charles Boole
99299
16
16
16
16
. . .
```

Each SET of information is called a **record**, while each piece of information in the record is called a **field**.

Example 2: To read records until the end of the file and output them:

```
with open('students.txt') as stu_file:

    # Read the first name
    name = stu_file.readline()

    while name != '':
        # Read the ID number
        id_num = stu_file.readline()

        # Strip the newlines from the fields.
        name = name.rstrip('\n')
        id_num = id_num.rstrip('\n')

        # Total a student's credit hours for 4 semesters.
        total = 0
        for x in range(4):
            hours = float(stu_file.readline())
            total += hours

        # Display the record.
        print(f'Name: {name}')
        print(f'ID: {id_num}')
        print(f'Total credits: {total}')
        print()

        # Read the name field of the next record.
        name = stu_file.readline()
```

Notes:

- The first name is read in then the code checks that a something has successfully been read BEFORE proceeding.
- Each time through the outer loop, the rest of the student's information is read in and processed.
- Before the end of the loop, attempt to read the next name but don't read further until you test that you're not at the end of the file.

It uses the order shown in Examples 1a&b. Why?

READ
TEST
PROCESS
READ NEXT
(loop back to test)

What if you wanted to store the credit hours in a list?

CSV Files

Plain text files use newline characters or other delimiters as separators between data items. A comma-separated values (CSV) file stores tabular data in a text file using commas to store multiple items on the same row. Each row is a record that ends in a newline character. The `csv` module is included in Python's standard library to make reading from and writing to CSV files easy and efficient. At the top of your code, import the module:

```
import csv
```

Example 3: Suppose you have a 2D list of values. To write data to a CSV file:

2D List to CSV

```
books = [["The Nightingale", 2015, "Kristin Hannah"],
         ["Teaching Students to Learn", 2015, "Saundra McGuire"],
         ["Hidden Figures", 2016, "Margot Lee Shetterly"]]

with open('booklist.csv', 'w', newline = '') as outfile:
    writer = csv.writer(outfile)
    writer.writerows(books)
```

Inspect the CSV file!

(use notebook provided)

To read data from a CSV file, use the `reader()` function to create a reader object, an iterator (an object that produces values one by one) and iterable (that produces an iterator) that yields a list of strings for each row in the file:

```
reader_object = csv.reader(file_object, delimiter='\t', encoding='utf-8')
```

Where only the file object must be specified, the other arguments are optional.

It's best to use the `newline = ''` argument when opening the file to enable universal newlines mode so that embedded newlines are correctly interpreted. By default, Python converts various newline characters to `'\n'`.

Example 4: To read in data from a CSV file:

CSV to Lists

```
with open('booklist.csv', 'r', newline = '') as infile:
    reader = csv.reader(infile)
    for row in reader:
        print(row)
```

iterate over reader

Prints each row of CSV as a list:

```
['The Nightingale', '2015', 'Kristin Hannah']
['Teaching Students to Learn', '2015', 'Saundra McGuire']
['Hidden Figures', '2016', 'Margot Lee Shetterly']
```

For more details on dialects and formatting: https://docs.python.org/3/library/csv.html#

Dictionary Reading and Writing

The `csv` module also allows for the creation of dictionary objects that map each row of the file to a dictionary using the values in the first row (column headings) as keys.

Example 5: To read in data from a CSV file with headings to create a dictionary:

```
with open('book_table.csv') as my_file:
    my_data = csv.DictReader(my_file)
    for line in my_data:
        print(line)
```

iterate over DictReader object

```
{'Title': 'The Nightingale', 'Year Published': '2015', 'Author': 'Kristin Hannah'}
{'Title': 'Teaching Students to Learn', 'Year Published': '2015', 'Author': 'Saundra McGuire'}
{'Title': 'Hidden Figures', 'Year Published': '2016', 'Author': 'Margot Lee Shetterly'}
```

where the `DictReader` object `my_data` is an iterable object with each item a dictionary.

Or you can format as you wish:

access as a dict

```
print(line['Title']+ 'by'+ line['Author'], line['Year Published'])
```

```
The Nightingale by Kristin Hannah 2015
Teaching Students to Learn by Saundra McGuire 2015
Hidden Figures by Margot Lee Shetterly 2016
```

Example 6: To create a CSV file with headings:

```
with open('book_csv.csv', 'w') as this_file:
    cols = ['Title', 'Year Published', 'Author']
    writer = csv.DictWriter(this_file, fieldnames=cols)
    writer.writeheader()

    writer.writerow({'Title': 'The Nightingale', 'Year Published': '2015',
            'Author': 'Kristin Hannah'}
    writer.writerow({'Title': 'Teaching Students to Learn',
            'Year Published': '2015', 'Author': 'Saundra McGuire'})
    writer.writerow({'Title': 'Hidden Figures', 'Year Published': '2016',
            'Author': 'Margot Lee Shetterly'})
```

Inspect the CSV file!

Demo with owid energy data (see owid_energy.ipynb)

https://github.com/owid/energy-data?tab=readme-ov-file
View codebook for details

Binary Files

Sometimes, we may want to use binary files. Why?

smaller, more compressed, faster since no conversion needed

Python provides the `pickle` module to automate the process of storing objects such as lists in a binary file.

Example 7: Use the dump method from the pickle module to write an object to a binary file:

```
import pickle
with open('book.bin', 'wb') as fout:
    pickle.dump(books, fout)
```

This is referred to as **serializing** or **pickling** the object.

Example 8: Suppose you have a 2D list of values. To read data from a CSV file:

```
with open('book.bin', 'rb') as fin:
     book_list = pickle.load(fin)
     print(book_list)
```

This is referred to as **de-serializing** or **un-pickling** the object.